AOSP 进阶丛书 · 第十七册 · ANDROID 13 (API 33)

无障碍体系源码精读

AccessibilityManagerService(:173)、AccessibilityInputFilter(:59) 输入截胡层、AccessibilityNodeInfo 语义快照协议、安全边界与 13 新限制。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：让所有人都能用 2
第一章 AccessibilityManagerService 2
第二章 AccessibilityInputFilter：输入链的截胡层 2
第三章 AccessibilityNodeInfo：无障碍快照协议 3
第四章 安全边界 4
第五章 设计决策六问 4
附录 A 观测 4
附录 B 快问快答（10 条） 5
附录 C 自检题 5
第六章 服务绑定链：从 meta-data 声明到事件通道就绪 5
第七章 事件管道：从 View.sendAccessibilityEvent 到服务回调 11
第八章 节点快照协议深入：seal、缓存与跨进程查找 18
第九章 代操作双通道：手势注入与 performAction 权限链 24
教科书习题集（分级） 29
术语表 32
排障手册：无障碍体系问题定位 34
无障碍核心速查表 36

本册速览

AccessibilityManagerService:173 extends IAccessibilityManager.Stub
AccessibilityServiceConnection：服务绑定与事件过滤树
AccessibilityInputFilter：书5 输入链的截胡层（可改写/吞事件）
AccessibilityNodeInfo：View 树的"无障碍快照"协议
安全边界：为什么无障碍权限极高（读屏+合成操作）

前言：让所有人都能用

无障碍（Accessibility）是 Android 的道德底线与工程挑战：视障用户靠 TalkBack 读屏、运动障碍用户靠开关控制操作——这些能力要求系统**在读屏之上再开一层语义快照协议**。本册解剖 AccessibilityManagerService、AccessibilityInputFilter（书5 输入链的截胡层）、AccessibilityNodeInfo 快照协议与服务安全边界。

源码基线不变。主角：`frameworks/base/services/accessibility/`（独立子目录，不在 services/core）。

第一章 AccessibilityManagerService

设计洞察 · 学习目标

1. 说出 AMS 在 SystemServer 中的启动阶段与三大核心职责；2. 描述 a11y 服务注册生命周期的四个阶段；3. 解释 meta-data 声明式过滤存在的理由；4. 用 dumpsys accessibility 读出绑定状态与事件计数

AccessibilityManagerService（:173 extends IAccessibilityManager.Stub）在 SystemServer 的 startOtherServices 阶段启动。核心职责三件：**注册管理**（哪些 a11y 服务已绑定/启用）、**事件分发**（窗口变化/文本变更事件按过滤树分发给订阅者）、**服务安全策略**（AccessibilitySecurityPolicy 检查哪些服务能看哪些事件）。

text a11y 服务的注册生命周期

```
1. 用户在设置开启 TalkBack →
2. AccessibilityManagerService.bindServices() →
AccessibilityServiceConnection 绑定 TalkBack 的 Service
3. 连接建立 → onServiceConnected 回调 → 服务开始收事件
4. 关闭 → unbind → 停止收事件
// 讲解注释：a11y 服务就是普通 Service（App 侧）——
// 额外的 <accessibility-service> meta-data 声明过滤与能力。
```

第二章 AccessibilityInputFilter：输入链的截胡层

设计洞察 · 学习目标

1. 定位 AccessibilityInputFilter 在书5 输入链中的位置；2. 区分 onKeyEvent 与 onMotionEvent 两条改写路径；3. 说明开关控制如何借 InputFilter 合成点击；4. 解释为什么截输入的特权只授予 a11y 体系

AccessibilityInputFilter（:59 extends InputFilter implements EventStreamTransformation）在书5 输入链的**最后一个消费者**位置——在 ViewRootImpl 的 InputStage 之后：

java services/accessibility/…/AccessibilityInputFilter.java:353-364（节选+讲解注释）

```
mEventHandler.get(Display.DEFAULT_DISPLAY).onKeyEvent(event, policyFlags);
```

```
// 讲解注释①：按键事件经 a11y 过滤链——服务可改写/吞掉/
// 合成新事件（如把点按转为长按）。
mEventHandler.get(...).onMotionEvent(transformedEvent, event, policyFlags);
// 讲解注释②：触摸事件同样可被改写——开关控制（Switch Access）
// 就在这里"扫描界面→用户按开关→合成点击"。
```

先修要求与学习路线

先修：已读第一册（架构总览）。

读完本书你将能够：

- 解释无障碍体系的核心机制与设计决策
- 追踪关键调用链并标注每次跨进程跳
- 使用 dumpsys/日志定位本领域问题
- 回答书7 对应章节的面试题

设计洞察 · 逐段精读：为什么 a11y 能截输入而其他服务不能

a11y 是系统授权的"代理操作者"——它的用户是**无法自己操作屏幕的人**，截输入是刚需。安全代价：恶意 a11y 服务=完全控制设备——13 起强制"设置里显式安装+弹窗确认"（禁止 sideload 后静默启用）。

第三章 AccessibilityNodeInfo：无障碍快照协议

设计洞察 · 学习目标

1. 解释快照而非直接引用 View 的两个理由；2. 说出快照携带的核心字段类别；3. 掌握自定义 View 的两个适配覆写点；4. 把快照协议与第八章的传输/缓存层区分开

a11y 服务不能直接操作 View 对象——它们通过 **AccessibilityNodeInfo**（View 树的"语义快照"）了解界面：

text 快照协议（读码归纳）

```
View.performAccessibilityAction() → View 生成 AccessibilityNodeInfo
包含：文本/类名/位置/可点击/可滚动/子节点列表…
→ a11y 服务读快照 → 找到"发送按钮"
→ 服务调 performAction(ACTION_CLICK) → 回到 View.performClick()
// 讲解注释：快照而非直接引用——跨进程安全+快照保证一致性
// （读到的界面状态在那一刻是完整的）。
```

自定义 View 的 a11y（开发视角）：覆写 `onInitializeAccessibilityNodeInfo` 与 `performAccessibilityAction`——不覆写则系统用默认信息（类名+位置），盲人用户"看不到"按钮含义。

第四章 安全边界

设计洞察 · 学习目标

1. 背出 a11y 特权的三层能力构成；2. 列出系统提供的三层防护及其代码落点；3. 解释能力位为何必须在安装期定型；4. 对照 13 代 sideload 限制说明威胁模型变化

无障碍权限极高的原因：**读屏（AccessibilityNodeInfo 全部内容）+ 合成操作（performAction 可替代用户点击）+ 输入截胡**。三层防护：安装来源限制（13 禁 sideload 静默启用）、用户显式授权弹窗、AccessibilitySecurityPolicy 按服务过滤事件类型。

第五章 设计决策六问

设计洞察 · 学习目标

1. 逐题复述六个设计决策的动机与代价；2. 把每个决策映射到对应源码机制；3. 用"刚需换风险"框架分析特权系统的边界；4. 对本册未覆盖的新问题（如 AI 读屏）套用同一分析框架

- **为什么 a11y 用快照而非直接操作 View？** 跨进程安全（服务在 App 进程外）+ 一致性保证（快照原子）；直接引用需要跨进程对象传递=Binder 全套开销且易竞态。
- **为什么 a11y 能截输入而其他服务不能？** 系统授权的代理操作者——刚需（用户无法自己操作）；安全代价已说。
- **为什么 a11y 服务声明在 meta-data 而非代码？** 系统需要在绑定前就知道"这个服务能做什么"（读屏/触摸探索/手势合成）——声明式过滤=安全审查的依据。
- **为什么 TalkBack 的"触摸探索"需要接管全部触摸？** 视障用户不知道按钮在哪——先触摸播报再双击确认=两步交互，第一步必须截获全部事件（否则被普通点击消费掉）。
- **为什么 13 禁止 sideload a11y？** a11y=读屏+全操作——恶意 sideload 的 a11y 服务等于后门（Google 的安全整改）。
- **为什么 a11y 事件按过滤树分发而非广播？** 事件量大（每次文本变化都触发）；过滤树按服务订阅范围裁剪——只发感兴趣的事件（省性能+隐私）。

附录 A 观测

目标	命令	看点	
已绑定服务	dumpsys accessibility	服务列表/绑定状态/事件计数	

目标	命令	看点	
无障碍节点	adb shell settings get secure enabled_accessibility_services	已启用	
TalkBack 状态	dumpsys accessibility |	grep -A5 TalkBack	是否运行

附录 B 快问快答（10 条）

- a11y 服务在哪跑？——App 进程（普通 Service），由 AccessibilityManagerService 绑定。
- 为什么权限极高？——读屏+合成操作+输入截胡。
- 13 的新安全限制？——禁止 sideload 静默启用 a11y。
- AccessibilityNodeInfo 是什么？——View 树的语义快照协议。
- 自定义 View 怎么适配？——覆写 onInitializeAccessibilityNodeInfo。
- 触摸探索怎么工作？——接管全部触摸：先触摸播报再双击。
- 开关控制（Switch Access）？——InputFilter 合成点击（:364 onMotionEvent）。
- a11y 事件按什么分发？——过滤树按服务订阅裁剪。
- InputFilter 在哪层？——书5 输入链的最后一个消费者。
- a11y 与书5 InputDispatcher 的关系？——InputDispatcher 分发到 a11y InputFilter→改写/转发。

附录 C 自检题

- 从 TalkBack 开启到读出"发送按钮"的全链（含 binder 跳点）。
- 为什么恶意 a11y 服务=后门（三层能力）？
- AccessibilityInputFilter 与 ViewRootImpl InputStage 的位置关系。

第六章 服务绑定链：从 meta-data 声明到事件通道就绪

> **设计洞察 · 学习目标**
> 1. 画出"安装→启用→绑定"三阶段中 AccessibilityServiceInfo 的产生路径；2. 背出 bindLocked 的四个绑定 flag 并解释每个 flag 的特权含义；3. 说出服务 off/binding/bound 三状态由哪两个集合维护；4. 追踪 onServiceConnected 到服务端 onServiceConnected 回调的完整跨进程链

一个 a11y 服务从"用户在设置里拨动开关"到"onAccessibilityEvent 里收到第一条事件"，中间隔着一条贯穿设置存储、系统服务、AMS、应用进程四个层次的绑定链。本章把这条链逐环拆开——它是第一章注册生命周期的源码级展开，也是第七章事件管道能够运转的前提：**没有连接，事件没有接**

收者。



图 1 服务绑定链时序：settings 表为目标态，updateServicesLocked 声明式收敛，bindLocked 五 flag 到 onServiceConnected

6.1 三个身份：已安装、已启用、已绑定

AMS 内部用三个彼此独立的状态刻画一个服务，混淆它们是排障时最常见的错误：

- **已安装（installed）**：包里有 Service 且 intent-filter 声明了 `android.accessibilityservice.AccessibilityService`。AMS 在用户启动/切换时用 `queryIntentServicesAsUser` 扫描出来（`AccessibilityManagerService.java:1836-1859`），为每个 ResolveInfo new 一个 `AccessibilityServiceInfo(resolveInfo, mContext)`——构造函数会解析 APK 的 meta-data XML（`<accessibility-service>` 标签），把 `canRetrieveWindowContent`、`canPerformGestures` 等布尔声明折算成能力位（`AccessibilityServiceInfo.java:689-713`）。**能力位在安装解析时定死，运行期不可伪造**——这是第四章安全边界的信任根。
- **已启用（enabled）**：组件名出现在 `Settings.Secure.enabled_accessibility_services` 冒号分隔的字符串里。SettingsObserver 监听到变化后走 `onUserStateChangedLocked`（`AccessibilityManagerService.java:2417`）。
- **已绑定（bound）**：`AccessibilityUserState.mBoundServices` 里存在活的 `AccessibilityServiceConnection`。它与 `mBindingServices`（`AccessibilityUserState.java:81-90`）共同维护服务三态：off、binding、bound。

注册前还有一道准入审查：`AccessibilitySecurityPolicy.canRegisterService`（`AccessibilitySecurityPolicy.java:657-690`）拒绝三类服务——没在 `<service>` 上声明

android.permission.BIND_ACCESSIBILITY_SERVICE 权限的（这等于绕过"只有系统有权绑定我"的保护）、标了 FLAG_EXTERNAL_SERVICE 的、被 AppOps OPSTR_BIND_ACCESSIBILITY_SERVICE 拉黑的。

6.2 绑定决策：updateServicesLocked

java services/accessibility/…/AccessibilityManagerService.java:2149-2204（骨架+讲解注释）

```
private void updateServicesLocked(AccessibilityUserState userState) {
Map<ComponentName, AccessibilityServiceConnection> componentNameToServiceMap =
userState.mComponentNameToServiceMap;
boolean isUnlockingOrUnlocked = LocalServices.getService(UserManagerInternal.class)
.isUserUnlockingOrUnlocked(userState.mUserId);
// 讲解注释①：直接启动（Direct Boot）阶段——用户尚未解锁、凭据加密存储
// 不可读，非 directBootAware 的服务此时一律跳过，解锁后再补绑。
for (int i = 0, count = userState.mInstalledServices.size(); i < count; i++) {
AccessibilityServiceInfo installedService = userState.mInstalledServices.get(i);
ComponentName componentName = ComponentName.unflattenFromString(
installedService.getId());
AccessibilityServiceConnection service = componentNameToServiceMap.get(componentName);
if (!isUnlockingOrUnlocked && !installedService.isDirectBootAware()) {
Slog.d(LOG_TAG, "Ignoring non-encryption-aware service " + componentName);
continue;
}
// 讲解注释②：binding 集合是"在途"标记——bindService 是异步的，
// 如果不挡住，同一服务的 onUserStateChanged 会被重复触发导致二次绑定。
if (userState.getBindingServicesLocked().contains(componentName)
|| userState.getCrashedServicesLocked().contains(componentName)) {
continue;
}
if (userState.mEnabledServices.contains(componentName)
&& !mUiAutomationManager.suppressingAccessibilityServicesLocked()) {
if (service == null) {
service = new AccessibilityServiceConnection(userState, mContext, componentName,
installedService, sIdCounter++, mMainHandler, mLock, mSecurityPolicy,
this, getTraceManager(), mWindowManagerService,
getSystemActionPerformer(), mA11yWindowManager,
mActivityTaskManagerService);
} else if (userState.mBoundServices.contains(service)) {
continue;
}
// 讲解注释③：只有"已启用 && 尚未绑定 && 不在崩溃黑名单"才走到这里。
// UiAutomation（instrumentation 测试用的特殊 a11y 客户端）接管时会
// 抑制普通服务，避免测试期被真实读屏服务干扰。
service.bindLocked();
} else {
if (service != null) {
// 讲解注释④：开关拨回 off 的分支——解绑并清掉无障碍按钮的快捷目标，
// 防止用户点击按钮去启动一个已失效的服务。
service.unbindLocked();
removeShortcutTargetForUnboundServiceLocked(userState, service);
}
}
}
final int count = userState.mBoundServices.size();
```

```
mTempIntArray.clear();
for (int i = 0; i < count; i++) {
final ResolveInfo resolveInfo =
userState.mBoundServices.get(i).mAccessibilityServiceInfo.getResolveInfo();
if (resolveInfo != null) {
mTempIntArray.add(resolveInfo.serviceInfo.applicationInfo.uid);
}
}
// 讲解注释⑤：把所有 a11y 服务的 uid 集合下发给 AudioManager 与 ATMS——
// 前者让"朗读音量"等通道识别 a11y 流量，后者用于 setAllowAppSwitches
// 的放行名单。绑定不只是"拉起进程"，更是系统各处的授权登记。
final AudioManagerInternal audioManager =
LocalServices.getService(AudioManagerInternal.class);
if (audioManager != null) {
audioManager.setAccessibilityServiceUids(mTempIntArray);
}
mActivityTaskManagerService.setAccessibilityServiceUids(mTempIntArray);
updateAccessibilityEnabledSettingLocked(userState);
}
```

注意这段代码是**声明式收敛**而非命令式拉起：方法不关心"用户刚拨了开关"还是"服务刚崩了要重启"，它只对比目标状态（enabled 集合）与当前状态（bound/binding/crashed 集合），把差值收敛到一致。任何触发 `onClientChangeLocked` 的路径（设置变更、包增删、崩溃重启、用户切换）最终都汇入这里。

6.3 bindLocked：一串非同寻常的绑定 flag

java services/accessibility/…/AccessibilityServiceConnection.java:105-127（骨架+讲解注释）

```
public void bindLocked() {
AccessibilityUserState userState = mUserStateWeakReference.get();
if (userState == null) return;
final long identity = Binder.clearCallingIdentity();
try {
int flags = Context.BIND_AUTO_CREATE
| Context.BIND_FOREGROUND_SERVICE_WHILE_AWAKE
// 讲解注释①：BIND_AUTO_CREATE——服务不存在则拉起进程；
// BIND_FOREGROUND_SERVICE_WHILE_AWAKE——亮屏期间按前台服务
// 优先级对待，TalkBack 不能因为内存回收而中途失聪。
| Context.BIND_ALLOW_BACKGROUND_ACTIVITY_STARTS
// 讲解注释②：允许后台启动 Activity——无障碍服务可以替盲人
// 打开设置页/对话框（performGlobalAction、打开通知等场景），
// 这正是第四章所说的"代理操作者"特权的绑定层体现。
| Context.BIND_INCLUDE_CAPABILITIES;
// 讲解注释③：BIND_INCLUDE_CAPABILITIES——把系统绑定方的凭证能力
// 传递给被绑进程，让服务回调里的 AttributionSource 带有足够权限
// 读取界面内容。
if (userState.getBindInstantServiceAllowedLocked()) {
flags |= Context.BIND_ALLOW_INSTANT;
}
if (mService == null && mContext.bindServiceAsUser(
mIntent, this, flags, new UserHandle(userState.mUserId))) {
userState.getBindingServicesLocked().add(mComponentName);
// 讲解注释④：mIntent 在构造函数里已 setComponent 指向目标服务，
```

```
// 并塞进 EXTRA_CLIENT_LABEL/EXTRA_CLIENT_INTENT——通知栏上
// "此应用正在使用无障碍服务"的跳转入口就是它。
// 绑定成功即登记"在途"，对应 6.2 的讲解注释②。
}
} finally {
Binder.restoreCallingIdentity(identity);
}
mActivityTaskManagerService.setAllowAppSwitches(mComponentName.flattenToString(),
mAccessibilityServiceInfo.getResolveInfo().serviceInfo.applicationInfo.uid,
userState.mUserId);
// 讲解注释⑤：把"这个 uid 允许切换后台 App"告诉 ATMS——TalkBack 聚焦到
// 另一个应用时，系统允许它把那个 App 切到前台。逐服务登记而非全局放行。
}
```

对照普通应用绑定：五个 flag 全是普通应用拿不到的系统能力（BIND_AUTO_CREATE 人人可用，其余都要求系统身份）。这解释了第五章"为什么 a11y 能截输入"的另一面——**特权从绑定这一刻就开始下发了**，而非等到事件回调。

6.4 onServiceConnected：双向登记完成

AMS 侧作为 ServiceConnection 的实现，被 SystemServer 主线程回调：

java services/accessibility/…/AccessibilityServiceConnection.java:170-207（骨架+讲解注释）

```
@Override
public void onServiceConnected(ComponentName componentName, IBinder service) {
synchronized (mLock) {
if (mService != service) {
if (mService != null) {
mService.unlinkToDeath(this, 0);
}
mService = service;
try {
mService.linkToDeath(this, 0);
// 讲解注释①：service 是 App 进程里 IAccessibilityServiceClient
// 包装器的 Binder 代理。linkToDeath 后服务进程崩溃会触发
// binderDied → 记入 crashed 集合 → updateServicesLocked 重绑，
// 这就是 a11y 服务"崩溃自动重启"的机制根源。
} catch (RemoteException re) {
Slog.e(LOG_TAG, "Failed registering death link");
binderDied();
return;
}
}
mServiceInterface = IAccessibilityServiceClient.Stub.asInterface(service);
AccessibilityUserState userState = mUserStateWeakReference.get();
if (userState == null) return;
userState.addServiceLocked(this);
// 讲解注释②：addServiceLocked（AccessibilityUserState.java:220）会调
// serviceConnection.onAdded() 并把连接放进 mBoundServices——
// 从这一刻起服务才算"已绑定"，绑定集合里的在途标记还要等
// initializeService 才清除。
mSystemSupport.onClientChangeLocked(false);
// Initialize the service on the main handler after we're done setting up for
```

```
// the new configuration (for example, initializing the input filter).
mMainHandler.sendMessage(obtainMessage(
AccessibilityServiceConnection::initializeService, this));
// 讲解注释③：initializeService 延后到主线程执行——此刻可能正在
// 重建输入过滤器（第二章），先让配置收敛，再通知客户端初始化。
if (requestImeApis()) {
mSystemSupport.requestImeLocked(this);
}
}
}
```

`initializeService`（AccessibilityServiceConnection.java:205-246）做三件事：把组件名从 binding/crashed 集合移出（在途期结束）、复核服务是否仍处于 enabled（期间被关掉就直接走解绑，不发 init）、最后 `serviceInterface.init(this, mId, mOverlayWindowTokens.get(DEFAULT_DISPLAY))` 把**连接 id** 与**覆盖窗口 token** 跨进程发给服务端。

6.5 App 侧：DO_INIT 与 onServiceConnected 回调

服务端最终收到的不是系统服务，而是自己应用里的 `IAccessibilityServiceClientWrapper`——binder 线程收到 `init` 后转投到服务主线程的消息队列：

java core/java/android/accessibilityservice/AccessibilityService.java:2803-2829（DO_INIT 骨架+讲解注释）

```
public void init(IAccessibilityServiceConnection connection, int connectionId,
IBinder windowToken) {
Message message = mCaller.obtainMessageIOO(DO_INIT, connectionId,
connection, windowToken);
mCaller.sendMessage(message);
// 讲解注释①：HandlerCaller 把 binder 线程的调用转成主线程消息——
// onServiceConnected 等生命周期回调因此永远跑在服务主线程，
// 开发者不需要自己处理线程安全。
}
// —— executeMessage 中的 DO_INIT 分支（同文件 :2840-2863）——
case DO_INIT: {
mConnectionId = message.arg1;
SomeArgs args = (SomeArgs) message.obj;
IAccessibilityServiceConnection connection =
(IAccessibilityServiceConnection) args.arg1;
IBinder windowToken = (IBinder) args.arg2;
args.recycle();
if (connection != null) {
AccessibilityInteractionClient.getInstance(mContext).addConnection(
mConnectionId, connection, /*initializeCache=*/true);
// 讲解注释②：把 AMS 侧连接的反向 Binder 代理登记进
// AccessibilityInteractionClient 的静态连接表——之后所有
// findAccessibilityNodeInfosByViewId / performAction 都凭
// mConnectionId 找到这条通路（第八章的主角）。
// initializeCache=true 同时创建该连接私有的 AccessibilityCache。
if (mContext != null) {
try {
connection.setAttributionTag(mContext.getAttributionTag());
} catch (RemoteException re) { /* ... */ }
}
mCallback.init(mConnectionId, windowToken);
```

```
mCallback.onServiceConnected();
// 讲解注释③：windowToken 是 AMS 分配的覆盖窗口 token——服务
// 想画高亮框/焦点框时用它 addView，画在所有应用之上。
// onServiceConnected 之后开发者才算可以调 getWindows 等接口。
} else {
AccessibilityInteractionClient.getInstance(mContext)
.clearCache(mConnectionId);
AccessibilityInteractionClient.getInstance(mContext).removeConnection(
mConnectionId);
mConnectionId = AccessibilityInteractionClient.NO_ID;
mCallback.init(AccessibilityInteractionClient.NO_ID, null);
// 讲解注释④：connection 为 null 的分支即解绑清理——连接表与
// 缓存同步移除，避免悬挂的 Binder 代理泄漏。
}
return;
}
```

`mCallback.onServiceConnected()` 最终走到 `dispatchServiceConnected`（`AccessibilityService.java:808-821`）：先通知各 MagnificationController 与 SoftKeyboardController"连接就绪"，再回调开发者覆写的 `onServiceConnected()`（:846）。至此闭环：**设置开关 → SettingsObserver → updateServicesLocked → bindServiceAsUser → AMS 收到 onServiceConnected → init 跨进程 → 服务端登记连接 + onServiceConnected 回调**。之后第七章的事件管道才有分发对象。

设计洞察 · 设计洞察：为什么绑定链要区分 binding 与 bound 两个集合

bindService 是异步的：AMS 发出绑定到收到 onServiceConnected 之间没有同步点。updateServicesLocked 会被设置变更、崩溃、包管理事件高频触发，若无 binding 集合做幂等屏障，每次触发都会对同一个服务重复 new Connection 并重复 bindService——bindServiceAsUser 对同一 ServiceConnection 的重复调用虽不会崩溃，但会打乱回调时序并让 crashed/binding 状态机失真。三集合（bound/binding/crashed）加声明式收敛，是系统服务处理"异步状态同步"的教科书样板，比 goto 式的"记住是谁触发了我"干净得多。

设计洞察 · 交叉链接

第一章的注册生命周期 text 图是本章的鸟瞰；第二章的输入过滤器在服务绑定完成后由 scheduleUpdateInputFilter 重建（输入截胡层的安装时机）；第八章解释 DO_INIT 里 initializeCache=true 创建的缓存如何随事件失效；第四章安全边界的能力位在本章 6.1 已落地为安装期常量。

第七章 事件管道：从 View.sendAccessibilityEvent 到服务回调

设计洞察 · 学习目标

1. 完整背出事件从 View 到 TalkBack 回调的六段跨层路径并标出三次跨进程跳；2. 解释 AMS 侧 canDispatch 与连接侧 wantsEvent 两道过滤各自把守什么；3. 说明 notificationTimeout 节流为何放行 TYPE_WINDOW_CONTENT_CHANGED；4. 追踪事件被 seal 与 setSource(null) 的降级时机

View 树的每一次点击、滚动、文本变化都可能变成一条 `AccessibilityEvent`。事件管道是第三层"语义广播"：应用进程产生事件，AMS 过滤后分发给各 a11y 服务。本章从生产端走到消费端，标出每一次跨进程跳，并回答两个关键问题——**一条事件凭什么被发出、凭什么被送到某个服务**。

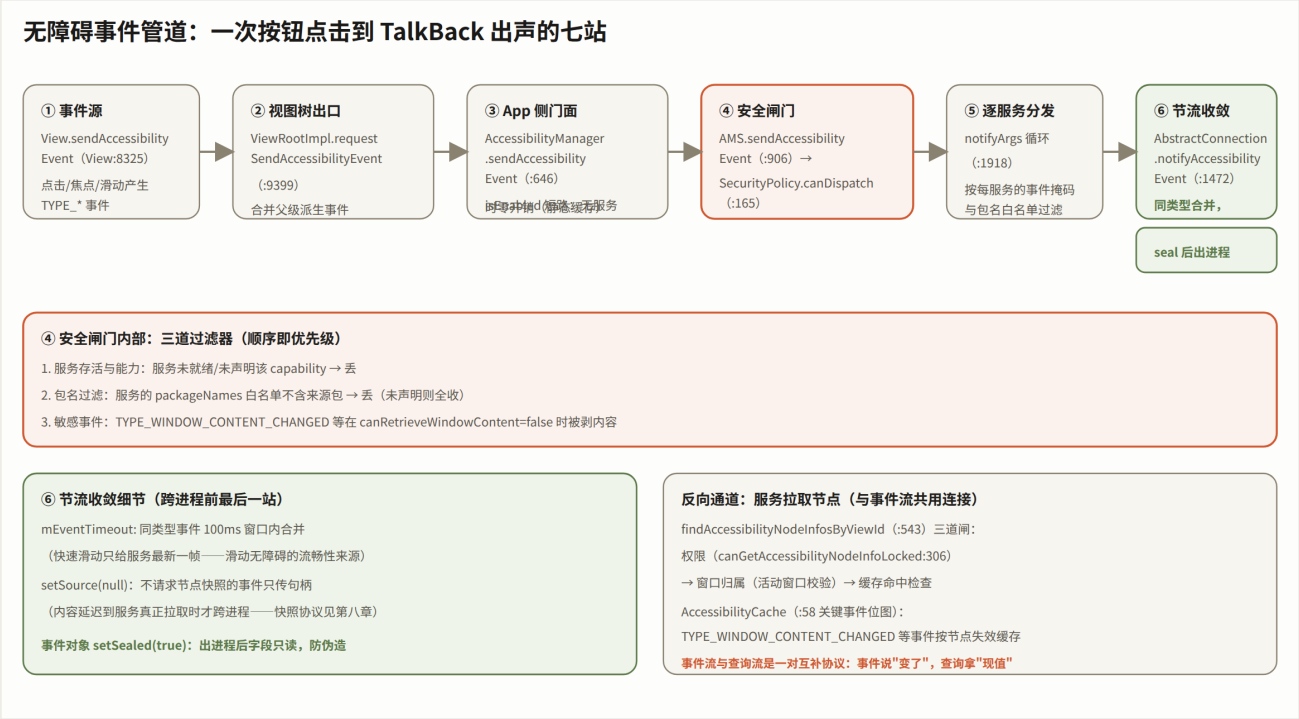


图 2 事件管道七站：View→ViewRootImpl→AccessibilityManager→AMS 安全闸门→逐服务分发→节流收敛，右下为反向节点查询通道

7.1 生产端：View 如何决定发事件

java core/java/android/view/View.java:8325-8434（骨架+讲解注释）

```
public void sendAccessibilityEvent(int eventType) {
if (mAccessibilityDelegate != null) {
mAccessibilityDelegate.sendAccessibilityEvent(this, eventType);
// 讲解注释①：delegate 优先——开发者 setAccessibilityDelegate 后
// 事件的产生与初始化全部交给 delegate，这是兼容库（如
// androidx.core.view.AccessibilityDelegateCompat）的挂钩点。
} else {
sendAccessibilityEventInternal(eventType);
}
}
// —— sendAccessibilityEventInternal :8363 检查 attach 状态后调用 ——
public void sendAccessibilityEventUncheckedInternal(AccessibilityEvent event) {
boolean isWindowStateChanged =
(event.getEventType() == AccessibilityEvent.TYPE_WINDOW_STATE_CHANGED);
```

```
boolean isWindowDisappearedEvent = isWindowStateChanged && ((event.getContentChangeTypes()
& AccessibilityEvent.CONTENT_CHANGE_TYPE_PANE_DISAPPEARED) != 0);
boolean detached = detached();
if (!isShown() && !isWindowDisappearedEvent && !detached) {
// 讲解注释②：不可见的 View 不发事件——屏幕外的控件变化对读屏
// 用户无意义，这里是事件量的第一道闸。例外是"面板消失"与
// 尚未 attach 的 View（后者走 postDelayed 兜底重发）。
return;
}
onInitializeAccessibilityEvent(event);
// 讲解注释③：把类名/包名/checked 等语义填进事件——自定义 View 适配
// 的另一挂点（第三章的 onInitializeAccessibilityNodeInfo 是快照侧
// 的对应物）。
if ((event.getEventType() & POPULATING_ACCESSIBILITY_EVENT_TYPES) != 0) {
dispatchPopulateAccessibilityEvent(event);
// 讲解注释④：只有少数事件类型才收集子 View 文本（TYPE_VIEW_
// CLICKED 等）——避免每次滚动都遍历整棵树。
}
SendAccessibilityEventThrottle throttle = getThrottleForAccessibilityEvent(event);
if (throttle != null) {
throttle.post(event);
// 讲解注释⑤：TYPE_VIEW_SCROLLED 有 View 级节流器——快速滚动的
// ListView 每帧都滚，事件却最多按固定间隔发一条。
} else if (!isWindowDisappearedEvent && detached) {
postDelayed(() -> { /* attach 后补发 */ },
ViewConfiguration.getSendRecurringAccessibilityEventsInterval());
} else {
requestParentSendAccessibilityEvent(event);
}
}
private void requestParentSendAccessibilityEvent(AccessibilityEvent event) {
ViewParent parent = getParent();
if (parent != null) {
getParent().requestSendAccessibilityEvent(this, event);
// 讲解注释⑥：事件沿 ViewParent 链逐层上抛（ViewGroup 逐级转发），
// 终点必然是 ViewRootImpl——它是唯一持有 AccessibilityManager
// 的树根，负责盖章窗口级信息后送出进程。
}
}
```

TYPE_WINDOW_CONTENT_CHANGED 走另一条汇聚路径：View 调 ViewRootImpl.postSendWindowContentChangedCallback（ViewRootImpl.java:9351-9357），由 SendWindowContentChangedAccessibilityEvent runnable（:10473）在 ViewConfiguration.getSendRecurringAccessibilityEventsInterval()（:792，常量 100ms）窗口内合并多条变化为一条。

7.2 树根盖章与第一次跨进程跳

java core/java/android/view/ViewRootImpl.java:9399-9441（骨架+讲解注释）

```
@Override
public boolean requestSendAccessibilityEvent(View child, AccessibilityEvent event) {
if (mView == null || mStopped || mPausedForTransition) {
return false;
```

```
}
// Immediately flush pending content changed event (if any) to preserve event order
if (event.getEventType() != AccessibilityEvent.TYPE_WINDOW_CONTENT_CHANGED
&& mSendWindowContentChangedAccessibilityEvent != null
&& mSendWindowContentChangedAccessibilityEvent.mSource != null) {
mSendWindowContentChangedAccessibilityEvent.removeCallbacksAndRun();
// 讲解注释①：顺序保证——一条点击事件要发出去时，先把挂起的
// "内容已变"事件立刻冲出去，否则读屏服务会先看到"被点击"
// 再看到"内容变了"，违反因果。
}
final int eventType = event.getEventType();
final View source = getSourceForAccessibilityEvent(event);
switch (eventType) {
case AccessibilityEvent.TYPE_VIEW_ACCESSIBILITY_FOCUSED: {
if (source != null) {
AccessibilityNodeProvider provider = source.getAccessibilityNodeProvider();
if (provider != null) {
final int virtualNodeId = AccessibilityNodeInfo.getVirtualDescendantId(
event.getSourceNodeId());
final AccessibilityNodeInfo node;
node = provider.createAccessibilityNodeInfo(virtualNodeId);
setAccessibilityFocus(source, node);
// 讲解注释②：虚拟节点（RecyclerView 条目等）的无障碍焦点
// 需要树根记账——之后 provider 派发绘制焦点框等逻辑
// 依赖这份记录。
}
}
} break;
case AccessibilityEvent.TYPE_WINDOW_CONTENT_CHANGED: {
handleWindowContentChangedEvent(event);
// 讲解注释③：合并同一 ViewRootImpl 内的多条内容变化事件——
// 每帧动画触发的 N 条事件收敛为一条。
} break;
}
mAccessibilityManager.sendAccessibilityEvent(event);
// 讲解注释④：第一次跨进程跳的前奏——AccessibilityManager 是进程单例，
// 它持有 IAccessibilityManager（AMS 的 Binder 代理）。
return true;
}
```

AccessibilityManager.sendAccessibilityEvent（AccessibilityManager.java:646-698）先做本地检查：event.setEventTime(SystemClock.uptimeMillis()) 盖时间戳；把 mPerformingAction（第八章会看到它由 performAction 路径设置）填进 event.getAction()；若主线程上无障碍已关闭则直接抛 IllegalStateException("Accessibility off. Did you forget to check that?")——这是开发者最常见的崩溃之一；最后 service.sendAccessibilityEvent(dispatchedEvent, userId)——**跨进程跳 #1**，事件进入 system_server。

7.3 AMS 侧：全局过滤与窗口账本

java services/accessibility/…/AccessibilityManagerService.java:906-1000（骨架+讲解注释）

```
public void sendAccessibilityEvent(AccessibilityEvent event, int userId) {
boolean dispatchEvent = false;
```

```
int resolvedUserId;
synchronized (mLock) {
if (event.getWindowId() ==
AccessibilityWindowInfo.PICTURE_IN_PICTURE_ACTION_REPLACER_WINDOW_ID) {
// The replacer window isn't shown to services. Move its events into the pip.
AccessibilityWindowInfo pip = mA11yWindowManager.getPictureInPictureWindowLocked();
if (pip != null) {
event.setWindowId(pip.getId());
// 讲解注释①：画中画的"动作替换器"是系统内部窗口，对服务
// 不可见——它的事件要归并进画中画窗口本身的 id。
}
}
resolvedUserId = mSecurityPolicy.resolveCallingUserIdEnforcingPermissionsLocked(userId);
// Make sure the reported package is one the caller has access to.
event.setPackageName(mSecurityPolicy.resolveValidReportedPackageLocked(
event.getPackageName(), UserHandle.getCallingAppId(), resolvedUserId,
getCallingPid()));
// 讲解注释②：防伪造——恶意 App 可以把 packageName 填成别人的包
// 名骗过服务的包名过滤；AMS 校验调用方 uid 是否真的拥有这个包，
// 不合法则改写成 null（服务侧空包名视为不过滤）。这是第四章
// "过滤树即隐私边界"的服务器端防线。
if (resolvedUserId == mCurrentUserId) {
if (mSecurityPolicy.canDispatchAccessibilityEventLocked(mCurrentUserId, event)) {
mA11yWindowManager.updateActiveAndAccessibilityFocusedWindowLocked(
mCurrentUserId, event.getWindowId(), event.getSourceNodeId(),
event.getEventType(), event.getAction());
mSecurityPolicy.updateEventSourceLocked(event);
dispatchEvent = true;
// 讲解注释③：canDispatch（AccessibilitySecurityPolicy.java:165）
// 把 WINDOW_STATE_CHANGED/通知/触摸探索/悬停等十类"全局
// 事件"无条件放行；内容变化类事件只有当存在能检索窗口
// 内容的服务时才放行——没有读屏服务在跑，App 侧虽然
// 发事件，system 侧直接丢弃（省 Binder 流量）。
}
if (mHasInputFilter && mInputFilter != null) {
mMainHandler.sendMessage(obtainMessage(
AccessibilityManagerService::sendAccessibilityEventToInputFilter,
this, AccessibilityEvent.obtain(event)));
// 讲解注释④：事件同时抄送输入过滤器——触摸探索的
// GestureManifold 靠 TYPE_TOUCH_INTERACTION_START/END 等
// 事件感知用户行为（第二章链路的事件入口）。
}
}
}
if (dispatchEvent) {
/* …计算 displayId；若 TYPE_WINDOW_STATE_CHANGED 且窗口信息滞后，
先调 WindowManagerInternal.computeWindowsForAccessibility 让
WMS 补齐窗口列表，必要时 postpone（讲解注释⑤）… */
synchronized (mLock) {
dispatchAccessibilityEventLocked(event);
}
}
if (OWN_PROCESS_ID != Binder.getCallingPid()) {
event.recycle();
// 讲解注释⑥：跨进程来的事件归 AMS 所有，分发完即回收进对象池；
```

```
// 本进程（UiAutomation 等）的事件由调用者自己回收。
}
}
private void dispatchAccessibilityEventLocked(AccessibilityEvent event) {
notifyAccessibilityServicesDelayedLocked(event, false);
notifyAccessibilityServicesDelayedLocked(event, true);
mUiAutomationManager.sendAccessibilityEventLocked(event);
// 讲解注释⑦：两趟遍历——非默认服务先、默认服务（isDefault，通常是
// 用户选定的主读屏服务）后；UiAutomation（测试框架）独立收全量。
}
```

notifyAccessibilityServicesDelayedLocked（AccessibilityManagerService.java:1918-1933）遍历 mBoundServices，按 service.mIsDefault == isDefault 二分后逐个调 service.notifyAccessibilityEvent(event)。注意它捕获 IndexOutOfBoundsException 直接 bail——服务可能正在遍历中被移除，这是"锁内遍历活列表"的实用妥协。

7.4 连接侧：wantsEvent 与节流

java services/accessibility/…/AbstractAccessibilityServiceConnection.java:1472-1512（骨架+讲解注释）

```
public void notifyAccessibilityEvent(AccessibilityEvent event) {
synchronized (mLock) {
final int eventType = event.getEventType();
final boolean serviceWantsEvent = wantsEventLocked(event);
final boolean requiredForCacheConsistency = mUsesAccessibilityCache
&& ((AccessibilityCache.CACHE_CRITICAL_EVENTS_MASK & eventType) != 0);
if (!serviceWantsEvent && !requiredForCacheConsistency) {
// 讲解注释①：两难局面的解法——服务没订阅这条事件，但它本地
// 缓存了节点树（第八章），关键事件不清缓存就会读到陈旧节点。
// 所以这类事件仍要送达，只是标记 serviceWantsEvent=false，
// 客户端只拿它刷新缓存、不回调 onAccessibilityEvent。
return;
}
if (!mSecurityPolicy.checkAccessibilityAccess(this)) {
// 讲解注释②：AppOps 层再查一次 OPSTR_ACCESS_ACCESSIBILITY——
// 用户在设置里撤销了"无障碍权限"的服务会在这里被静默断流。
return;
}
AccessibilityEvent newEvent = AccessibilityEvent.obtain(event);
// 讲解注释③：复制一份——AMS 那份要分发给多个服务，各服务可能
// 因权限不同被降级改写（见下方 internal），不能共享同一实例。
Message message;
if ((mNotificationTimeout > 0)
&& (eventType != AccessibilityEvent.TYPE_WINDOW_CONTENT_CHANGED)) {
// Allow at most one pending event
final AccessibilityEvent oldEvent = mPendingEvents.get(eventType);
mPendingEvents.put(eventType, newEvent);
if (oldEvent != null) {
mEventDispatchHandler.removeMessages(eventType);
oldEvent.recycle();
// 讲解注释④：notificationTimeout 节流（值来自服务 XML 声明的
// android:notificationTimeout）：同类型事件只保留最新一条，
// 旧消息撤销、旧事件回收——TalkBack 每秒最多收到 N 条同类
// 事件，远端打印日志的服务不会被内容变化风暴淹没。
```

```
}
message = mEventDispatchHandler.obtainMessage(eventType);
} else {
// Send all messages, bypassing mPendingEvents
message = mEventDispatchHandler.obtainMessage(eventType, newEvent);
// 讲解注释⑤：TYPE_WINDOW_CONTENT_CHANGED 不节流——它已被
// 应用侧（ViewRootImpl 100ms 合并）与 wantsEvent 双重过滤，
// 再节流会破坏缓存一致性事件的时效。
}
message.arg1 = serviceWantsEvent ? 1 : 0;
mEventDispatchHandler.sendMessageDelayed(message, mNotificationTimeout);
}
}
```

wantsEventLocked（:1521-1545）是逐服务过滤树的核心：三连判——服务可收事件（非崩溃、非挂起）；事件源 View 对无障碍重要（isImportantForAccessibility），除非服务声明了 FLAG_INCLUDE_NOT_IMPORTANT_VIEWS；事件类型与 mEventTypes（XML accessibilityEventTypes）掩码全匹配（注意是 != eventType 的**全含**判断，掩码部分匹配不算命中）；包名在 mPackageNames（XML packageNames）白名单内或白名单为空。

7.5 发车与落地：seal、降级、回调

java services/accessibility/…/AbstractAccessibilityServiceConnection.java:1548-1631（骨架+讲解注释）

```
private void notifyAccessibilityEventInternal(
int eventType, AccessibilityEvent event, boolean serviceWantsEvent) {
IAccessibilityServiceClient listener;
synchronized (mLock) {
listener = mServiceInterface;
if (listener == null) {
return;
// 讲解注释①：消息在 handler 队列里等待的窗口期，服务可能已经
// 解绑/崩溃——死引用检查必须放在节流延迟之后。
}
if (event == null) {
/* 节流路径：从 mPendingEvents 取同类型最新事件（注释列出了
6 步竞态推演：消息取出后、锁获得前，另一 binder 线程可能
换掉事件并回收旧的——所以必须锁内重取并判空） */
event = mPendingEvents.get(eventType);
if (event == null) return;
mPendingEvents.remove(eventType);
}
if (mSecurityPolicy.canRetrieveWindowContentLocked(this)) {
event.setConnectionId(mId);
// 讲解注释②：connectionId 是事件与连接表的关联键——服务收到
// 事件后调 event.getSource() 时，AccessibilityInteractionClient
// 凭它反查 AMS 连接去拉快照（第八章）。
} else {
event.setSource((View) null);
// 讲解注释③：没有 CAPABILITY_CAN_RETRIEVE_WINDOW_CONTENT 的
// 服务收到的事件被摘除 source 指针——它能知道"发生了什么"，
// 但无法反查界面内容。能力位检查在此从"承诺"变成"执行"。
}
event.setSealed(true);
```

```
// 讲解注释④：封缄——事件跨越最后一道进程边界前被标记只读，
// 防止服务改写事件字段后污染其他观察者或伪造回传。
}
try {
listener.onAccessibilityEvent(event, serviceWantsEvent);
// 讲解注释⑤：跨进程跳 #2——oneway binder 调用直达服务进程，
// AMS 不等服务返回（否则一个慢服务就能拖住整个分发循环）。
} catch (RemoteException re) {
Slog.e(LOG_TAG, "Error during sending " + event + " to " + listener, re);
} finally {
event.recycle();
}
}
```

服务进程侧收尾（AccessibilityService.java:2878-2895 的 DO_ON_ACCESSIBILITY_EVENT）：事件先喂给 AccessibilityInteractionClient.onAccessibilityEvent（刷新本地缓存，无论 serviceWantsEvent 与否），再按 serviceWantsEvent 决定是否回调开发者 onAccessibilityEvent(event)，最后 event.recycle()。**事件管道全程对象池化**：obtain/recycle 贯穿两侧进程，任何一环忘记回收都是系统级内存抖动。

设计洞察 · 设计洞察：为什么事件过滤做两遍（AMS 全局一遍、连接逐服务一遍）

第一遍（canDispatch）回答"这个事件值得进 system_server 的分发循环吗"——无服务需要时直接丢弃，Binder 反序列化是事件管道最大的固定成本；第二遍（wantsEvent）回答"这个服务订阅了吗"——每服务一套事件类型/包名掩码，互相独立。两遍的粒度不同导致一个微妙现象：没有任何服务订阅 TYPE_VIEW_TEXT_CHANGED 时，App 侧 sendAccessibilityEvent 仍照常发出、AMS 侧照常走完 canDispatch 判定（文本变化不在全局白名单，此时会因"无检索内容服务"被拦截）——两层过滤不是冗余，而是成本递进的漏斗。

设计洞察 · 交叉链接

事件管道的终点连着第八章：serviceWantsEvent=false 的"仅缓存一致性事件"如何驱动 AccessibilityCache 失效，是快照协议正确性的关键；第七章 6.4 的 connectionId 与第三章的快照协议在此合流——event.getSource() 的懒加载第一次跨进程跳就发生在那章的 getRootInActiveWindow 链路里。

第八章 节点快照协议深入：seal、缓存与跨进程查找

设计洞察 · 学习目标

1. 解释节点快照为何只传节点 ID 引用而非整棵子树；2. 背出 findAccessibilityNodeInfosByViewId 从服务到目标 App 的四段路径与三道权限闸；3. 说出 AccessibilityCache 的失效矩阵（哪类事件清哪个粒度）；4. 理解 seal 封缄如何让"读到的快照"不可变不可伪造

第三章建立了快照协议的概念模型；本章进入协议的**传输与一致性**层。核心矛盾：TalkBack 每秒可能要读几十个节点的属性，而每个属性都住在另一个进程的 View 对象里——Android 用"快照 + 节点 ID 懒加载 + 客户端缓存 + 事件驱动失效"四件套解决，这是整个无障碍体系最精巧的分布式设计。

8.1 协议三要素：ID、快照、连接表

- **节点 ID**：64 位 `makeNodeId(accessibilityViewId, virtualDescendantId)`（`AccessibilityNodeInfo.java:845`）——高 32 位是 View 在窗口内的序号，低 32 位是虚拟节点号（RecyclerView 条目等）。ID 是"引用"，不是"实体"。
- **快照（AccessibilityNodeInfo）**：单个节点的属性包（文本/类名/边界/动作位/父 ID/子 ID 列表）。parcel 化时子节点**只写 ID 列表**（`AccessibilityNodeInfo.java:3954` 取 `mChildNodeIds` 序列化），子节点的属性按需另拉——一次 IPC 只换一棵"骨架"。
- **连接表**：第六章 DO_INIT 登记的 `sCaches`（`AccessibilityInteractionClient.java:121`，静态 `SparseArray<AccessibilityCache>`）与连接代理，connectionId 是表键。服务代码里的 `event.getSource()`、`info.getChild(i)` 全都靠 connectionId 反查通路。

8.2 按 ViewId 查找：AMS 侧的中枢

TalkBack 找"发送按钮"的典型路径是 `findAccessibilityNodeInfosByViewId`。服务进程把请求交给 AMS，AMS 做完全部安全判定后**把请求转投给目标 App 进程**：

java services/accessibility/…/AbstractAccessibilityServiceConnection.java:543-610（骨架+讲解注释）

```
public String[] findAccessibilityNodeInfosByViewId(int accessibilityWindowId,
long accessibilityNodeId, String viewIdResName, int interactionId,
IAccessibilityInteractionConnectionCallback callback, long interrogatingTid)
throws RemoteException {
final int resolvedWindowId;
RemoteAccessibilityConnection connection;
Region partialInteractiveRegion = Region.obtain();
synchronized (mLock) {
mUsesAccessibilityCache = true;
// 讲解注释①：服务一旦做过节点查询，就永久进入"缓存一致性"阵营——
// 第七章讲解注释①的 requiredForCacheConsistency 由此位开启。
if (!hasRightsToCurrentUserLocked()) {
return null;
// 讲解注释②：闸 1——调用方必须属于当前用户（或系统/ROOT/Shell，
// 见 AccessibilityServiceConnection.java:267 的覆写：SHELL_UID
// 放行正是 adb 命令能 dump 节点的原因）。
}
resolvedWindowId = resolveAccessibilityWindowIdLocked(accessibilityWindowId);
// 讲解注释③：ACTIVE_WINDOW_ID(-1) 等魔法值在此翻译成真实窗口 id，
// 并核对窗口是否对服务可见（隐藏窗口直接解析失败）。
final boolean permissionGranted =
mSecurityPolicy.canGetAccessibilityNodeInfoLocked(
mSystemSupport.getCurrentUserIdLocked(), this, resolvedWindowId);
if (!permissionGranted) {
return null;
// 讲解注释④：闸 2——canGetAccessibilityNodeInfoLocked
//  （AccessibilitySecurityPolicy.java:306）= 有
// CAPABILITY_CAN_RETRIEVE_WINDOW_CONTENT 能力位 且 目标窗口
// isRetrievalAllowingWindowLocked（IME/系统敏感窗口有额外
// 白名单规则）。
} else {
connection = mA11yWindowManager.getConnectionLocked(
```

```
mSystemSupport.getCurrentUserIdLocked(), resolvedWindowId);
if (connection == null) {
return null;
// 讲解注释⑤：窗口 id → 目标 App 进程连接的映射表在
// AccessibilityWindowManager——AMS 自己不持有任何 View，
// 它只是"接线员"：把服务进程的 callback 与目标 App 的
// IAccessibilityInteractionConnection 配对。
}
}
if (!mA11yWindowManager.computePartialInteractiveRegionForWindowLocked(
resolvedWindowId, partialInteractiveRegion)) {
partialInteractiveRegion.recycle();
partialInteractiveRegion = null;
// 讲解注释⑥：部分交互区域——窗口被遮挡/超出屏幕的部分不可点，
// 预计算成 Region 传给 App 侧，让快照带上"哪些区域真正可交互"。
}
}
final Pair<float[], MagnificationSpec> transformMatrixAndSpec =
getWindowTransformationMatrixAndMagnificationSpec(resolvedWindowId);
if (!mSecurityPolicy.checkAccessibilityAccess(this)) {
return null;
// 讲解注释⑦：闸 3——AppOps 层（第七章 7.4 的同一检查）。
}
final int interrogatingPid = Binder.getCallingPid();
callback = replaceCallbackIfNeeded(callback, resolvedWindowId, interactionId,
interrogatingPid, interrogatingTid);
// 讲解注释⑧：画中画窗口的回调要换成 ActionReplacingCallback——PiP
// 菜单里的动作要映射回宿主 Activity 的节点。
try {
connection.getRemote().findAccessibilityNodeInfosByViewId(accessibilityNodeId,
viewIdResName, partialInteractiveRegion, interactionId, callback, mFetchFlags,
interrogatingPid, interrogatingTid, spec, transformMatrix);
// 讲解注释⑨：跨进程跳 #2——AMS 把"查询 + 回调 + 询问方身份
//  （interrogatingPid/Tid，App 侧据此决定是否跳过请求预备器）"
// 一起转给目标 App 进程，然后立刻返回 packageNames。
return mSecurityPolicy.computeValidReportedPackages(
connection.getPackageName(), connection.getUid());
} catch (RemoteException re) {
/* … */
} finally {
Binder.restoreCallingIdentity(identityToken);
if (partialInteractiveRegion != null && Binder.isProxy(connection.getRemote())) {
partialInteractiveRegion.recycle();
}
}
return null;
}
```

8.3 App 侧接应：UI 线程遍历与回调闭合

目标 App 进程里，`ViewRootImpl.AccessibilityInteractionConnection`（`ViewRootImpl.java:10301`，`IAccessibilityInteractionConnection.Stub`）在 binder 线程收到请求后，转手交给 `AccessibilityInteractionController`（`AccessibilityInteractionController.java:`

74）：

- `findAccessibilityNodeInfoByAccessibilityIdClientThread`（:169）把请求打成 Message，`scheduleMessage` 排程——若询问方就是本进程 UI 线程则直接执行（免死锁），否则 post 到 UI 线程，并排入 `AccessibilityRequestPreparer`（高优先级请求预备）队列。
- UI 线程执行真正的树遍历：从目标 View 起 `createAccessibilityNodeInfo`，按 `mFetchFlags` 与 prefetch 策略**连带预取子孙节点**（一次遍历产出一批快照，摊薄 IPC 成本），然后把结果经 `IAccessibilityInteractionConnectionCallback.setFindAccessibilityNodeInfosResult` **回抛给服务进程**——注意这是第三次跨进程跳，方向反转。
- 服务进程的 `AccessibilityInteractionClient` 早在请求处阻塞等待：`getFindAccessibilityNodeInfosResultAndClear(interactionId)` 用 interactionId 匹配回调（:718 附近），超时或 id 不匹配则返回空——**同步阻塞 + 异步回调闭合**的模板。

8.4 收尾三件套：finalize、seal、入缓存

java core/java/android/view/accessibility/AccessibilityInteractionClient.java:1262-1287（骨架+讲解注释）

```
private void finalizeAndCacheAccessibilityNodeInfo(AccessibilityNodeInfo info,
int connectionId, boolean bypassCache, String[] packageNames) {
if (info != null) {
info.setConnectionId(connectionId);
// Empty array means any package name is Okay
if (!ArrayUtils.isEmpty(packageNames)) {
CharSequence packageName = info.getPackageName();
if (packageName == null
|| !ArrayUtils.contains(packageNames, packageName.toString())) {
info.setPackageName(packageNames[0]);
// 讲解注释①：包名防伪的最后一环——computeValidReportedPackages
//  （AMS 返回值）给出目标 uid 名下的合法包名集合；快照包名
// 不在集合内就强制改写。服务看到的 packageName 一定可信。
}
}
info.setSealed(true);
// 讲解注释②：封缄。sealed 之后所有 setter 走 enforceNotSealed()
//  （AccessibilityNodeInfo.java:1235 等）抛 IllegalStateException，
// 而 getChild/getParent 等查询走 enforceSealed()（:1009）要求
// 已封缄——"写前必未封、读前必已封"，双向断言让服务进程拿到
// 的快照成为不可变的事实记录，杜绝"改一个字段再回传伪造界面"。
if (!bypassCache) {
AccessibilityCache cache = getCache(connectionId);
if (cache == null) { /* ... */ return; }
cache.add(info);
// 讲解注释③：连同预取的整批快照一起入缓存（sCaches 按
// connectionId 隔离）——下次 getChild(i) 若命中缓存就不再
// 发 IPC。FLAG_PREFETCH_* 策略决定预取深度。
}
}
}
```

parcel 化还有一处 13 代的工程细节值得一提：`writeToParcelNoRecycle`（`AccessibilityNodeInfo.java:3805`）不再逐字段写 parcel，而是先用一个 long 位图记录"哪些字段

偏离 DEFAULT 实例"，再只写非默认字段——View 属性绝大多数是默认值（没有 text、不可点击），字段压缩通常省一半以上 parcel 空间。读者可对照第三章：**协议演进的方向不是加字段，而是让常用 路径更便宜**。

8.5 AccessibilityCache：失效矩阵

缓存的价值取决于失效的正确性。失效的驱动源正是第七章送达的事件——连接侧把 `serviceWantsEvent=false` 的"仅缓存一致性事件"也发过来，客户端 `AccessibilityInteractionClient.onAccessibilityEvent` 转投 `AccessibilityCache.onAccessibilityEvent`：

java core/java/android/view/accessibility/AccessibilityCache.java:58-69, 206-288（骨架+讲解注释）

```
public static final int CACHE_CRITICAL_EVENTS_MASK =
AccessibilityEvent.TYPE_VIEW_ACCESSIBILITY_FOCUSED
| AccessibilityEvent.TYPE_VIEW_ACCESSIBILITY_FOCUS_CLEARED | AccessibilityEvent.TYPE
_VIEW_FOCUSED
| AccessibilityEvent.TYPE_VIEW_SELECTED | AccessibilityEvent.TYPE_VIEW_TEXT_CHANGED
| AccessibilityEvent.TYPE_VIEW_CLICKED | AccessibilityEvent.TYPE_VIEW_TEXT_SELECTION
_CHANGED
| AccessibilityEvent.TYPE_WINDOW_CONTENT_CHANGED | AccessibilityEvent.TYPE_VIEW_SCRO
LLED
| AccessibilityEvent.TYPE_WINDOWS_CHANGED | AccessibilityEvent.TYPE_WINDOW_STATE_CHA
NGED;
// 讲解注释①：关键事件掩码——AMS 侧 requiredForCacheConsistency 用的就是这个集合（第七章
7.4），两侧必须一致。
public void onAccessibilityEvent(AccessibilityEvent event) {
AccessibilityNodeInfo nodeToRefresh = null;
synchronized (mLock) {
if (!mEnabled) { return; }
switch (event.getEventType()) {
case AccessibilityEvent.TYPE_VIEW_ACCESSIBILITY_FOCUSED: {
// 讲解注释②：无障碍焦点迁移——焦点节点属性已变旧快照作废；源码换焦点前还会先清旧焦点（骨架省略）。
mAccessibilityFocus = event.getSourceNodeId(); mAccessibilityFocusedWindow = event.g
etWindowId();
nodeToRefresh = removeCachedNodeLocked(mAccessibilityFocusedWindow, mAccessibilityFo
cus);
} break;
case AccessibilityEvent.TYPE_VIEW_SELECTED:
case AccessibilityEvent.TYPE_VIEW_TEXT_CHANGED:
case AccessibilityEvent.TYPE_VIEW_CLICKED:
case AccessibilityEvent.TYPE_VIEW_TEXT_SELECTION_CHANGED: {
// 讲解注释③：节点自身状态变化——只失效事件源节点这一个粒度，移出的交给 refresher 重拉回填。
nodeToRefresh = removeCachedNodeLocked(event.getWindowId(), event.getSourceNodeId())
;
} break;
case AccessibilityEvent.TYPE_WINDOW_CONTENT_CHANGED: {
// 讲解注释④：事件自带变化范围声明——声明了 CONTENT_CHANGE_TYPE_SUBTREE
就整棵剪掉，否则仅失效源节点。
if ((event.getContentChangeTypes() & AccessibilityEvent.CONTENT_CHANGE_TYPE_SUBTREE)
!= 0) {
clearSubTreeLocked(event.getWindowId(), event.getSourceNodeId());
} else {
```

```
nodeToRefresh = removeCachedNodeLocked(event.getWindowId(), event.getSourceNodeI
d());
}
} break;
case AccessibilityEvent.TYPE_VIEW_SCROLLED: {
// 讲解注释⑤：滚动=位置全变，清子树（最粗的常规失效档）。
clearSubTreeLocked(event.getWindowId(), event.getSourceNodeId());
} break;
case AccessibilityEvent.TYPE_WINDOWS_CHANGED:
if (event.getWindowChanges() == AccessibilityEvent.WINDOWS_CHANGE_ACCESSIBILITY_FOCU
SED) {
// 讲解注释⑥：仅焦点窗口变化不清节点，只清"窗口列表"缓存。
clearWindowCacheLocked();
break;
} // 注意 fall-through
case AccessibilityEvent.TYPE_WINDOW_STATE_CHANGED: {
// 讲解注释⑦：窗口状态翻转（切应用/弹对话框）=世界重建，全缓存清空（最保守也最正确的一档）。
mValidWindowCacheTimeStamp = event.getEventTime();
clear();
} break;
}
}
if (nodeToRefresh != null) {
// 讲解注释⑧：失效即刷新——被移除的节点立刻重新拉取并回填，把"失效"与"再拉取"合并成一次 IPC。
if (mAccessibilityNodeRefresher.refreshNode(nodeToRefresh, true)) {
add(nodeToRefresh);
}
}
}
```

失效矩阵总结：**单节点失效**（focus/selected/text/clicked 事件）→ **子树失效**（SUBTREE 内容变化、滚动）→ **窗口列表失效**（WINDOWS_CHANGE_ACCESSIBILITY_FOCUSED）→ **全清**（WINDOW_STATE_CHANGED、其他 WINDOWS_CHANGED）。粒度从事件里"白拿"（`getContentChangeTypes/getWindowChanges`），这就是为什么第三章强调自定义 View 发事件时应声明精确的变化类型——**它不只是给读屏服务的提示，更是给缓存的失效指令**。

> **设计洞察 · 设计洞察：为什么缓存放在服务进程而不是 AMS**
>
> 同样一份数据，放 AMS 只能省一次 IPC 中转（查询仍要到 App 遍历），放服务进程则让最常见的"getChild 连续读"完全本地化。代价是每个服务各持一份副本、一致性要靠事件流驱动——而事件管道（第七章）恰好保证关键事件必达（requiredForCacheConsistency 绕过 wantsEvent 过滤）。缓存位置的选择暴露了这个体系的取舍偏好：宁可每个服务多占内存，也不让高热读路径挤占 system_server 的 binder 线程。UiAutomation（测试）走 bypassCache 路径，因为测试要求绝对新鲜的界面状态——同一协议、两种一致性档位。

设计洞察 · 交叉链接

第六章 DO_INIT 的 initializeCache=true 在这里兑现为 sCaches 的按连接隔离；第七章 7.4 的 requiredForCacheConsistency 与本章 CACHE_CRITICAL_EVENTS_MASK 是同一常量的两端；第四章的"读屏即看穿一切"在本章落地为三道闸（用户/能力位+窗口白名单/AppOps）与两个改写点（包名归一、seal 封缄）。

第九章 代操作双通道：手势注入与 performAction 权限链

设计洞察 · 学习目标

1. 区分三条代操作通道（performAction/performGlobalAction/dispatchGesture）的语义层级与各自权限闸；2. 解释注入手势为何会被真实触摸取消以及这个设计的动机；3. 说出 MotionEventInjector 在输入过滤器链头的位置及其后果；4. 追踪 performAction 从服务到目标 View 的完整路径并指出 action 的"盖章回流"

读屏服务替用户"按下"一个按钮有两条路：**语义级**的 `performAction(ACTION_CLICK)`——走第八章的快照协议通道，请求目标 App 自己执行点击逻辑；**物理级**的 `dispatchGesture`——把一段合成轨迹注入 InputDispatcher，走第二章的输入过滤器链路。前者像"替你按遥控器"，后者像"替你伸手指"。本章把两条链都走通，并对照它们的权限检查。

9.1 通道一：dispatchGesture 客户端拆步

java core/java/android/accessibilityservice/AccessibilityService.java:1160-1191（骨架+讲解注释）

```
public final boolean dispatchGesture(@NonNull GestureDescription gesture,
@Nullable GestureResultCallback callback,
@Nullable Handler handler) {
final IAccessibilityServiceConnection connection =
AccessibilityInteractionClient.getInstance(this).getConnection(mConnectionId);
if (connection == null) {
return false;
// 讲解注释①：连接没就绪（onServiceConnected 之前调用）直接失败——
// gesture API 不排队、不缓存，状态机比事件回调严格。
}
int sampleTimeMs = calculateGestureSampleTimeMs(gesture.getDisplayId());
List<GestureDescription.GestureStep> steps =
MotionEventGenerator.getGestureStepsFromGestureDescription(gesture, sampleTimeMs);
// 讲解注释②：客户端先"拆步"——StrokeDescription 描述的路径被按显示
// 器刷新率采样成一串 GestureStep（每步含该时间片的触摸点集合）。
// R 之前固定 100ms 采样，R 起按刷新率（60Hz≈16ms）保证滑动顺滑。
try {
synchronized (mLock) {
mGestureStatusCallbackSequence++;
if (callback != null) {
if (mGestureStatusCallbackInfos == null) {
mGestureStatusCallbackInfos = new SparseArray<>();
}
```

```
GestureResultCallbackInfo callbackInfo = new GestureResultCallbackInfo(gesture,
callback, handler);
mGestureStatusCallbackInfos.put(mGestureStatusCallbackSequence, callbackInfo);
// 讲解注释③：回调不随请求传递——客户端只发一个自增序号，
// 结果回来时按序号回查。oneway binder 的异步结果靠这层
// 序号配对，与第八章 interactionId 是同一手法。
}
connection.dispatchGesture(mGestureStatusCallbackSequence,
new ParceledListSlice<>(steps), gesture.getDisplayId());
// 讲解注释④：跨进程跳 #1——步进列表一次性打包过 AMS。
// 注意 displayId 是参数而非隐含值：多屏设备上服务可以
// 指定往哪块屏注入。
}
} catch (RemoteException re) {
throw new RuntimeException(re);
}
return true;
}
```

9.2 AMS 侧：能力位检查与注入器路由

java services/accessibility/…/AccessibilityServiceConnection.java:455-483（骨架+讲解注释）

```
@Override
public void dispatchGesture(int sequence, ParceledListSlice gestureSteps, int displayId) {
synchronized (mLock) {
if (mServiceInterface != null && mSecurityPolicy.canPerformGestures(this)) {
// 讲解注释①：闸 1——canPerformGestures
//  （AccessibilitySecurityPolicy.java:357）只查一个能力位
// CAPABILITY_CAN_PERFORM_GESTURES（XML canPerformGestures，
// 安装期解析，见第六章 6.1）。
MotionEventInjector motionEventInjector =
mSystemSupport.getMotionEventInjectorForDisplayLocked(displayId);
// 讲解注释②：按目标显示器路由注入器——每个显示器一个
// MotionEventInjector 实例（AccessibilityInputFilter.java:499
// 创建、AMS 经 setMotionEventInjectors 登记，:1475）。
if (motionEventInjector != null
&& mWindowManagerService.isTouchOrFaketouchDevice()) {
motionEventInjector.injectEvents(
gestureSteps.getList(), mServiceInterface, sequence, displayId);
// 讲解注释③：设备必须支持触摸或假触摸——无任何触屏的
// 设备（电视盒子接遥控器）手势注入无从谈起，直接失败。
} else {
mServiceInterface.onPerformGestureResult(sequence, false);
// 讲解注释④：失败也要回执——客户端 onPerformGestureResult
// 收到 false 才知道手势没执行（否则调用方永远等待）。
}
}
}
}
```

9.3 注入器：延迟调度与真实输入打断

java services/accessibility/…/MotionEventInjector.java:183-237（骨架+讲解注释）

```
private void injectEventsMainThread(List<GestureStep> gestureSteps,
IAccessibilityServiceClient serviceInterface, int sequence, int displayId) {
if (mIsDestroyed) {
serviceInterface.onPerformGestureResult(sequence, false);
return;
// 讲解注释①：显示器关闭/过滤器重建后注入器可能已销毁。
}
if (getNext() == null) {
notifyService(serviceInterface, sequence, false);
return;
// 讲解注释②：链上没有下游（输入过滤器未安装完整链）则拒绝。
}
boolean continuingGesture = newGestureTriesToContinueOldOne(gestureSteps);
if (continuingGesture) {
if ((serviceInterface != mServiceInterfaceForCurrentGesture)
|| !prepareToContinueOldGesture(gestureSteps)) {
cancelAnyPendingInjectedEvents();
notifyService(serviceInterface, sequence, false);
return;
// 讲解注释③：多指手势支持"续步"——第二次调用若与在途手势
// 接得上（如先按住再拖第二根手指），就并入当前手势而非
// 重建；但换了个服务来续、或时间戳接不上，就整个失败。
}
}
if (!continuingGesture) {
cancelAnyPendingInjectedEvents();
cancelAnyGestureInProgress(EVENT_SOURCE);
// 讲解注释④：新手势开始前先清场——未发的旧步进作废（补发
// ACTION_CANCEL 给下游），在途手势也被取消。
}
mServiceInterfaceForCurrentGesture = serviceInterface;
long currentTime = SystemClock.uptimeMillis();
List<MotionEvent> events = getMotionEventsFromGestureSteps(gestureSteps,
(mSequencesInProgress.size() == 0) ? currentTime : mLastScheduledEventTime);
if (events.isEmpty()) {
notifyService(serviceInterface, sequence, false);
return;
}
mSequencesInProgress.add(sequence);
for (int i = 0; i < events.size(); i++) {
MotionEvent event = events.get(i);
event.setDisplayId(displayId);
int isEndOfSequence = (i == events.size() - 1) ? 1 : 0;
Message message = mHandler.obtainMessage(
MESSAGE_SEND_MOTION_EVENT, isEndOfSequence, 0, event);
mLastScheduledEventTime = event.getEventTime();
mHandler.sendMessageDelayed(message, Math.max(0, event.getEventTime() - currentTime));
// 讲解注释⑤：核心——不是一次性把整串 MotionEvent 灌进输入链，
// 而是按事件时间戳 sendMessageDelayed 逐个"按拍"发出。60Hz
// 采样的 300ms 滑动会得到约 18 个延时消息，App 侧看到的是
// 时间上真实的滑动轨迹，而非瞬时传送。
```

```
}
}
@Override
public void onMotionEvent(MotionEvent event, MotionEvent rawEvent, int policyFlags) {
/* …鼠标悬停移动在注入进行中可豁免（:129-134 的特判）… */
cancelAnyPendingInjectedEvents();
// 讲解注释⑥：真实触摸到来 → 未发送的注入步进立刻作废并发
// ACTION_CANCEL。设计动机：屏幕是用户的最后自留地——当真人的
// 手指落在屏上，代操作必须让位，否则一个失控的读屏服务能和
// 用户"抢屏幕"互相覆盖输入。
policyFlags |= WindowManagerPolicyConstants.FLAG_INJECTED_FROM_ACCESSIBILITY;
// 讲解注释⑦：标记注入来源——App 侧 EventLog 与指纹识别依赖此位，
// 恶意软件分析时能区分"用户真点"与"a11y 合成点击"。
sendMotionEventToNext(event, rawEvent, policyFlags);
}
```

sendMotionEventToNext（MotionEventInjector.java:306）调 super.onMotionEvent 把事件交给链上**下一个**变换器。结合第二章与 AccessibilityInputFilter.enableFeaturesForDisplay（AccessibilityInputFilter.java:440-500）的 addFirstEventHandler 装配顺序（autoclick → TouchExplorer → 放大手势 → 注入器依次插到链头），最终链序是：**MotionEventInjector → MagnificationGestureHandler → TouchExplorer/Autoclick → 输出**。两个后果值得记：注入事件会先经过放大处理器做坐标变换（放大状态下注入也能点准）；触摸探索的 GestureManifold 能看到注入事件（所以注入与触摸探索互斥时段要靠 TouchExplorer 的 passthrough 状态协调）。

9.4 通道二：performAction 的权限链与闭环

语义级代操作复用第八章的查询通路，只是请求从"读快照"换成"执行动作"：

- **服务进程**：info.performAction(ACTION_CLICK, args) → AccessibilityInteractionClient.performAccessibilityAction → connection 代理。
- **AMS 侧**：AbstractAccessibilityServiceConnection.performAccessibilityAction（:941-977）——三道闸与 8.2 完全同构：hasRightsToCurrentUserLocked → canGetAccessibilityNodeInfoLocked（能力位 + 窗口白名单）→ checkAccessibilityAccess（AppOps）。**能读才能操作：能力位是读的门槛，也是写的门槛**，这就是第四章"读屏即全权"在代码里的体现。
- **目标 App 进程**：ViewRootImpl.AccessibilityInteractionConnection.performAccessibilityAction（ViewRootImpl.java:10337）→ AccessibilityInteractionController.performAccessibilityActionClientThread（:739，转 UI 线程消息）→ performAccessibilityActionUiThread（:760）：

java core/java/android/view/AccessibilityInteractionController.java:771-797（骨架+讲解注释）

```
boolean succeeded = false;
try {
if (mViewRootImpl.mView == null || mViewRootImpl.mAttachInfo == null ||
mViewRootImpl.mStopped || mViewRootImpl.mPausedForTransition) {
return;
// 讲解注释①：App 自身状态审查——窗口移除、停止或转场动画中
// 不执行代操作（此时执行点击可能落在过期界面上）。
```

```
}
mViewRootImpl.mAttachInfo.mAccessibilityFetchFlags = flags;
final View target = findViewByAccessibilityId(accessibilityViewId);
if (target != null && isShown(target)) {
// 讲解注释②：isShown 复检——用户看不见的控件不允许被代点击，
// 与第七章 View 侧"不可见不发事件"是同一条原则的写侧镜像。
mA11yManager.notifyPerformingAction(action);
// 讲解注释③：盖章回流——把正在执行的动作记进 App 进程的
// AccessibilityManager.mPerformingAction（:1145）。之后 App 发出
// 的下一条事件若 action 为 0，会被盖章为该动作
//  （AccessibilityManager.java:659-661），读屏服务由此知道
// "界面变化是我刚才那次点击引起的"。因果链靠两个系统状态
// 字段接力，没有显式回执。
if (action == R.id.accessibilityActionClickOnClickableSpan) {
succeeded = handleClickableSpanActionUiThread(target, virtualDescendantId,
arguments);
// 讲解注释④：富文本超链接点击是隐藏动作，需要先定位
// ClickableSpan 再构造点击坐标，单独处理。
} else {
AccessibilityNodeProvider provider = target.getAccessibilityNodeProvider();
if (provider != null) {
succeeded = provider.performAction(virtualDescendantId, action, arguments);
} else if (virtualDescendantId == AccessibilityNodeProvider.HOST_VIEW_ID) {
succeeded = target.performAccessibilityAction(action, arguments);
}
// 讲解注释⑤：虚拟节点走 provider，宿主走 View——第三章的
// "自定义 View 适配 performAccessibilityAction"就是最后这行。
}
mA11yManager.notifyPerformingAction(0);
}
} finally {
/* …构造结果经 callback.sendAccessibilityActionResult 回抛服务进程，
交互 id 配对与 8.3 相同… */
}
```

第三条通道 performGlobalAction（AbstractAccessibilityServiceConnection.java:972-978）最简单：只查 hasRightsToCurrentUserLocked，然后转交 SystemActionPerformer（系统返回/通知栏/最近任务等全局动作，权限由各动作的注册方自审）——**不查能力位**，因为全局动作不触碰任何应用界面内容。

> **设计洞察 · 设计洞察：为什么手势注入要"逐拍延时"而不一次性灌入**
>
> 合成轨迹如果瞬间送达，App 侧在一次 vsync 内收到 ACTION_DOWN 到 ACTION_UP 的全部事件，long-click 检测、惯性滑动、缩放手势全部失效——手势的语义住在时间轴里。sendMessageDelayed 按事件时间戳逐拍重放，让下游 InputDispatcher 与 App 按真实节奏消费；而"真实输入取消注入"（讲解注释⑥）则是反向的时间权：用户的实时性永远高于服务的计划性。这两条合起来体现了注入子系统的立场——它模拟的是"一个存在的人"，不是"一串坐标"。

设计洞察 · 交叉链接

注入链的前半段（InputDispatcher → AccessibilityInputFilter）与第二章互补——那章讲"截胡与改写"，本章讲"合成与插入"；performAction 的查询通路是第八章 8.2/8.3 的动作版；performGlobalAction 交由 SystemActionPerformer 执行的"系统动作"清单可对照第一章 AMS 的 SystemActionPerformer 成员。三条通道的能力位全部在第六章 6.1 的 XML 解析处定型。

教科书习题集（分级）

第一级：简答题（检验概念理解）

- **1.1 a11y 服务运行在哪个进程？由谁负责绑定？**

参考答案要点：运行在服务自己的 App 进程（普通 Service + meta-data 声明）；由 system_server 的 AccessibilityManagerService 创建 AccessibilityServiceConnection 并 bindServiceAsUser，绑定 flag 带多项系统特权。

- **1.2 AccessibilityNodeInfo 是什么？为什么用快照而非直接引用 View？**

参考答案要点：View 树的跨进程语义快照（文本/类名/位置/动作位/父子节点 ID）。直接引用需跨进程对象传递，Binder 开销大且易竞态；快照原子一致且可 seal 封缄防篡改。

- **1.3 bindLocked 设置了哪些非默认绑定 flag？各自授予什么能力？**

参考答案要点：BIND_FOREGROUND_SERVICE_WHILE_AWAKE（亮屏期间前台优先级防回收）、BIND_ALLOW_BACKGROUND_ACTIVITY_STARTS（允许替用户后台拉起界面）、BIND_INCLUDE_CAPABILITIES（传递系统能力凭证）、可选 BIND_ALLOW_INSTANT（绑定即时应用）。

- **1.4 服务三状态 off/binding/bound 由哪两个集合维护？为什么要单独维护 binding？**

参考答案要点：AccessibilityUserState 的 mBoundServices 与 mBindingServices（外加 mCrashedServices）。bindService 异步，无 binding 屏障则每次 onUserStateChanged 都会重复 new Connection 并重复绑定。

- **1.5 事件从 View 到服务回调要经过哪六段？标出跨进程跳。**

参考答案要点：View.sendAccessibilityEvent → 沿 ViewParent 链上抛到 ViewRootImpl.requestSendAccessibilityEvent → AccessibilityManager → 跨进程跳一（binder 到 AMS）→ AMS 全局过滤/盖章后逐服务 notifyAccessibilityEvent → 节流后跨进程跳二（oneway binder 到服务进程）→ HandlerCaller 转主线程回调 onAccessibilityEvent。

- **1.6 wantsEventLocked 的三连判是什么？**

参考答案要点：服务可收事件（非崩溃/挂起）；事件源 isImportantForAccessibility（或服务声明 FLAG_INCLUDE_NOT_IMPORTANT_VIEWS）；事件类型被 mEventTypes 掩码全含且包名在 mPackageNames 白名单（或白名单为空）。

- **1.7 CACHE_CRITICAL_EVENTS_MASK 为什么包含 TYPE_WINDOW_CONTENT_CHANGED 与 TYPE_VIEW_SCROLLED?**

参考答案要点：两者都意味着缓存节点属性可能过期（内容/位置变化），服务即使没订阅该事件也必须收到以维持缓存一致性——AMS 侧 requiredForCacheConsistency 依此放行"仅缓存事件"。

- **1.8 seal 封缄的双向断言指什么?**

参考答案要点：写侧——所有 setter 调 enforceNotSealed，已封缄则抛 IllegalStateException；读侧——getChild/getParent 等查询调 enforceSealed，要求已封缄。保证服务拿到的快照是不可变的事实记录。

- **1.9 三条代操作通道各自的权限闸是什么?**

参考答案要点：performAction——hasRightsToCurrentUser + canGetAccessibilityNodeInfoLocked（能力位+窗口白名单）+ AppOps checkAccessibilityAccess；dispatchGesture——canPerformGestures 能力位 + 触摸设备检查 + 注入器存在；performGlobalAction——仅 hasRightsToCurrentUser，动作本身由 SystemActionPerformer 注册方自审。

- **1.10 为什么真实触摸会取消注入中的手势?**

参考答案要点：屏幕是用户的最后自留地——MotionEventInjector.onMotionEvent 收到真实事件即 cancelAnyPendingInjectedEvents 并向下游补发 ACTION_CANCEL，保证用户的实时输入永远优先于服务的计划输入，防止失控服务与用户抢屏幕。

第二级：分析题（检验机制理解）

- **2.1 调用链追踪**：用户开启 TalkBack 后点击"发送"按钮，TalkBack 收到 TYPE_VIEW_CLICKED 并读出按钮文本。画出全链并标注：三次跨进程跳、seal 的时机、事件与快照两条通路的交汇点。

参考答案要点：点击事件走第七章管道（App→AMS→TalkBack）；读文本走第八章快照链（TalkBack→AMS→App→回抛）。交汇在 connectionId：事件盖章 connectionId，快照凭 connectionId 反查通路；seal 发生在 AMS 侧 notifyAccessibilityEventInternal 与服务侧 finalizeAndCache 两处。

- **2.2 失败推演（缓存漂移）**：假设 AMS 侧 requiredForCacheConsistency 被误删，只保留 serviceWantsEvent 过滤。TalkBack 界面上会出现什么症状？为什么?

参考答案要点：TalkBack 不订阅的事件（如 TYPE_VIEW_TEXT_SELECTION_CHANGED）不再驱动其 AccessibilityCache 失效，服务会读到陈旧节点——焦点框位置错、朗读旧文本；因为缓存失效完全依赖关键事件必达，删掉兜底通道等于砍断一致性。

- **2.3 失败推演（注入被取消）**：服务注入一段 500ms 滑动，第 200ms 用户手指落下。逐事件推演 InjectedPending/下游 InputDispatcher/App 侧各自的行为。

参考答案要点：onMotionEvent 收到真实 ACTION_DOWN → cancelAnyPendingInjectedEvents 作废剩余延时消息 → 向下游补发 ACTION_CANCEL 终止注入序

列 → App 收到半截滑动+CANCEL，惯性不触发；服务侧 onPerformGestureResult 收到失败回执。

- **2.4 跨册串联**：指出本册三次跨进程跳在书3 Binder 十跳中的位置与线程分布（binder 线程池 vs 主线程消息队列）。

参考答案要点：跳一（App→AMS 事件上报）落在 AMS binder 线程、持 mLock 短临界区；跳二（AMS→服务 onAccessibilityEvent）是 oneway，进服务 HandlerCaller 主线程队列；跳三（AMS→目标 App 查询/动作 + 回抛）binder 线程收、UI 线程执行，同步方在服务主线程阻塞等 interactionId。

- **2.5 对比分析**：从时序、原子性、权限、可取消性四个维度列表对比 performAction 与 dispatchGesture。

参考答案要点：时序——performAction 在目标 App UI 线程同步执行后回执，dispatchGesture 按时间轴逐拍重放；原子性——前者单动作原子，后者长时序可中断；权限——前者"能读才能写"，后者独立能力位；可取消性——后者可被真实输入/新手势取消，前者不可（但有 isShown 复检）。

- **2.6 一致性分析**：缓存失效四档粒度（单节点/子树/窗口列表/全清）分别由什么事件触发？为什么 WINDOW_STATE_CHANGED 必须全清？

参考答案要点：单节点——focus/selected/text/clicked 类；子树——SUBTREE 内容变化与 VIEW_SCROLLED；窗口列表——WINDOWS_CHANGE_ACCESSIBILITY_FOCUSED；全清——WINDOW_STATE_CHANGED 与其他 WINDOWS_CHANGED。窗口状态翻转意味着窗口集合与节点 ID 空间整体换代，任何局部保留都可能命中已死窗口的旧 ID。

- **2.7 机制归因**：TalkBack 进程被杀后为什么用户几乎无感恢复？画出 binding→crashed→重绑的状态机，指出各迁移的触发点。

参考答案要点：linkToDeath 触发 binderDied → 记入 mCrashedServices 并 unbind → onClientChangeLocked → updateServicesLocked 对"已启用但未绑定"的服务重新 bindLocked → 新连接 onServiceConnected → initializeService 从 crashed 集合移除并置 crashed=false。用户感知到的只有短暂静音。

第三级：设计题

- **3.1 架构演进（跨设备读屏）**：若要求把手机界面快照协议投递给平板上的读屏服务，协议需要引入哪些机制？给出分层方案。

参考答案要点：至少三件事——窗口/节点 ID 的跨设备命名空间（设备号并入 ID 或 per-device 连接表）；一致性模型的延迟重估（事件驱动的失效在跨设备链路上可能乱序/迟到，需要序号或租约）；权限模型的扩展（AppOps 与 capability 的跨设备映射与用户确认链）。

- **3.2 最小权限设计**：为"只朗读不代操作"的语音助手设计 a11y 服务声明，说明每个字段的最小化依据，并指出系统侧哪两处机制保证"声明即边界"。

参考答案要点：canRetrieveWindowContent=true（必须读）、canPerformGestures/canRequestFilterKeyEvents/canControlMagnification 均缺省 false、packageNames 限定自有应用、notificationTimeout 适度节流。系统侧保证：能力位安装期解析后

运行期不可伪造（第六章），AMS 侧每个查询/动作闸都复检能力位（第八、九章），事件按掩码裁剪（第七章）。

- **3.3 分层排障方案**：设计"服务收不到事件"的三层定位方案，每层给出命令与判据。

参考答案要点：第一层服务侧——确认 onServiceConnected 已回调、检查 onInterrupt/onAccessibilityEvent 是否被覆写遗漏；第二层系统侧——`adb shell dumpsys accessibility` 看 bound 列表与事件计数是否增长、`adb shell settings get secure enabled_accessibility_services` 确认组件名精确匹配；第三层应用侧——确认事件源 View isShown 且 importantForAccessibility、事件类型在服务 XML 订阅掩码内（对应第七章两道过滤的复现）。

术语表

术语	解释
a11y 服务三状态	服务在 AMS 视角的 off、binding、bound 三态，由 mBoundServices、mBindingServices、mCrashedServices 三个集合共同维护
AccessibilityCache	服务进程内按 connectionId 隔离的节点快照缓存，靠关键事件驱动失效，粒度分单节点、子树、窗口列表、全清四档
AccessibilityInputFilter	位于书5 输入链上的系统输入过滤器，把触摸与按键事件交给触摸探索、放大、注入等变换器链处理
AccessibilityInteractionClient	服务进程内持有连接表与缓存的查询端，负责发起跨进程节点查询、配对 interactionId 回执并 finalize 快照
AccessibilityManagerService	无障碍体系的系统服务（AMS），负责服务绑定、事件分发与安全策略，在 SystemServer startOtherServices 阶段启动
AccessibilityServiceConnection	AMS 侧每个已启用服务对应一个的连接对象，实现 ServiceConnection，负责 bind、登记与死亡重绑
AccessibilitySecurityPolicy	AMS 的安全策略组件：注册准入（canRegisterService）、节点读取权限、事件可分发判定与 AppOps 审查
AppOps 访问审查	以 OPSTR_ACCESS_ACCESSIBILITY 与 OPSTR_BIND_ACCESSIBILITY_SERVICE 两个操作位在注册与事件/查询路径上的动态放行或断流
bindLocked 特权绑定 flag	绑定 a11y 服务时的一组系统级 flag：前台服务优先级、后台启动 Activity、能力凭证传递、可选绑定即时应用
binding 在途集合	bindService 发出到 onServiceConnected 之间的幂等屏障集合，防止状态收敛循环重复绑定同一服务
CACHE_CRITICAL_EVENTS_MASK	缓存关键事件掩码，覆盖焦点、内容、滚动、窗口变化等事件；AMS 用它把"仅缓存一致性"事件强制送达未订阅的服务

术语	解释
canRegisterService	注册准入检查：必须声明 BIND_ACCESSIBILITY_SERVICE 权限、禁止 FLAG_EXTERNAL_SERVICE、通过 AppOps 黑名单
capability 能力位	安装期从 meta-data XML 解析出的能力比特（读窗口内容、过滤按键、执行手势等），运行期不可伪造，是所有操作闸的依据
connectionId	AMS 分配的连接编号，贯穿事件盖章（setConnectionId）与服务侧快照查询的反查，是两条通路的关联键
Direct Boot 感知	用户解锁前（凭据加密存储不可读）只有声明 directBootAware 的服务才会被绑定
dispatchGesture	物理级代操作通道：把 GestureDescription 拆成按刷新率采样的步进序列，经 AMS 注入输入链按时间轴重放
FLAG_INJECTED_FROM_ACCESSIBILITY	注入事件携带的策略标记，用于把"a11y 合成输入"与用户真实输入区分开
getRootInActiveWindow	获取活动窗口根节点快照的标准入口，ACTIVE_WINDOW_ID 等魔法值在 AMS 侧翻译为真实窗口
interactionId	服务侧自增的交互序号，用于在同步阻塞的查询中配对异步回调结果，超时或错配返回空
makeNodeId 节点编码	64 位节点 ID：高 32 位为 View 序号、低 32 位为虚拟节点号，是快照协议中的节点引用
MotionEventInjector	每显示器一个的注入器，按事件时间戳逐拍发送合成触摸；真实触摸到来即作废待发事件并补发 ACTION_CANCEL
notificationTimeout 通知节流	服务 XML 声明的同类型事件最小间隔：节流路径每类型只保留最新一条事件，TYPE_WINDOW_CONTENT_CHANGED 除外
partialInteractiveRegion	窗口可交互部分的预计算区域，随查询传给目标 App，让快照带上遮挡与屏幕外信息
performAction 权限链	语义级代操作的三道闸：当前用户校验、能力位加窗口白名单、AppOps 审查；能读才能写
seal 封缄	快照与事件的只读标记：写前必未封（enforceNotSealed）、读前必已封（enforceSealed），保证跨进程事实不可篡改
setAllowAppSwitches	绑定/解绑时向 ATMS 登记服务 uid 的后台切换放行，让读屏服务能把目标应用切到前台
TouchExplorer	触摸探索实现：接管全部触摸，先播报焦点再双击确认，位于输入过滤器变换器链中
wantsEventLocked	连接侧逐服务订阅过滤：可收事件、重要性、事件类型掩码全含、包名白名单四连判

术语	解释
覆盖窗口 token	AMS 在 init 时下发给服务的窗口 token，服务用它把焦点框等覆盖层画在所有应用之上
节点快照懒加载	parcel 只携带子节点 ID 引用而非子节点实体，子树属性按需逐次拉取以摊薄 IPC 成本
事件双级节流	View 侧 SendAccessibilityEventThrottle 与 ViewRootImpl 100ms 合并、连接侧 notificationTimeout 节流构成的两级流量整形
无障碍事件	View 树状态变化的语义广播（AccessibilityEvent），经生产、盖章、过滤、节流、封缄五步抵达订阅服务

排障手册：无障碍体系问题定位

通用定位路径（自上而下收窄）：1. **状态层**——`adb shell dumpsys accessibility` 看已绑定服务、能力位、事件计数是否增长；2. **配置层**——`adb shell settings get secure enabled_accessibility_services` 核对启用清单、`adb shell settings get secure accessibility_enabled` 确认总开关；3. **日志层**——`adb logcat -s AccessibilityManagerService` 捕获绑定与分发日志；4. **机制层**——按第七章两道过滤、第八章三道闸逐层归因。

深度实战：无障碍体系的排障场景分析

场景：TalkBack 读不出自定义 View

1. 检查 View.setImportantForAccessibility

2. 覆写 onInitializeAccessibilityNodeInfo → 设置 text/contentDescription

3. 覆写 performAccessibilityAction → 支持 ACTION_CLICK

4. 用 Accessibility Scanner 验证

5. 自检：dumpsys accessibility → 服务是否收到事件

场景一：服务已启用却始终未绑定

症状：设置里显示服务已开启，但 `adb shell dumpsys accessibility` 的 bound services 列表里没有它，功能整体无效。

定位命令：`adb shell settings get secure enabled_accessibility_services` 核对组件名是否与包名+全限定类名完全一致；`adb logcat -s AccessibilityManagerService` 查找 "Skipping accessibility service" 前缀的拒绝日志；**adb shell cmd appops get 包名 BIND_ACCESSIBILITY_SERVICE** 查 AppOps 拉黑。

根因：三类常见——组件名写错（相对类名 `.Svc` 只有在包名一致时才解析）；`<service>` 缺少 `android.permission.BIND_ACCESSIBILITY_SERVICE` 权限声明，被 canRegisterService 拒绝（

AccessibilitySecurityPolicy.java:657）；设备处于 Direct Boot 阶段且服务未声明 directBootAware （AccessibilityManagerService.java:2162 检查）。

修复：写出全限定 ComponentName；补权限声明；等待用户解锁或声明 directBootAware；被 AppOps 拉黑则 **adb shell cmd appops set 包名 BIND_ACCESSIBILITY_SERVICE allow** 复位后重启用服务。

场景二：服务已绑定但 onAccessibilityEvent 不触发

症状：dumpsys 显示服务在 bound 列表，但事件计数不增长，回调永远不来。

定位命令：`adb shell dumpsys accessibility` 反复执行对比该服务的 event 计数（AMS 侧收到没收到）；**adb shell dumpsys package 包名** 核对 meta-data 中 accessibilityEventTypes 与 packageNames；对可疑 View 打印 `isShown()` 与 `isImportantForAccessibility()`。

根因：按第七章两道过滤归因——事件类型不在 mEventTypes 掩码内（注意是全含判断，多类型订阅要用 OR 组合值）；包名白名单排除了目标应用；事件源 View 不可见（View.java:8406 的 isShown 短路）；notificationTimeout 节流把连续事件合并成一条（看似"丢事件"实为设计行为）；服务未声明 FLAG_INCLUDE_NOT_IMPORTANT_VIEWS 而 View 被标记 not important。

修复：放宽掩码与包名做二分定位；给事件源 View 补语义或加 flag；接受节流语义（提高事件质量的正道是精简订阅而非关节流）。

场景三：getRootInActiveWindow 返回 null（节点树为空）

症状：服务能收到事件，但拉快照全为 null 或空列表，TalkBack 表现为"读得出有界面变化、读不出内容"。

定位命令：`adb shell dumpsys accessibility` 确认该服务条目带有 retrieveWindowContent 能力描述；`adb logcat -s AccessibilityManagerService` 查 "Error findAccessibilityNodeInfoByViewId" 与 "No connection"；确认前台是否是输入法等敏感窗口。

根因：XML 未声明 canRetrieveWindowContent（能力位缺失，canGetAccessibilityNodeInfoLocked 直接拒绝，AccessibilitySecurityPolicy.java:306）；目标窗口不在 isRetrievalAllowingWindow 白名单（IME/系统敏感界面）；AppOps 的 OPSTR_ACCESS_ACCESSIBILITY 被用户撤销；目标进程正在销毁窗口（连接表中连接已移除）。

修复：补能力位声明后重新启用服务；对敏感窗口走系统应用白名单流程；检查设置里"无障碍权限"是否被关闭；对瞬态窗口加重试而非视为故障。

场景四：performAction 返回 false（代操作失败）

症状：服务已拿到快照，调 `performAction(ACTION_CLICK)` 却返回 false，或抛 IllegalStateException。

定位命令：`adb logcat -s AccessibilityManagerService`；在目标 View 上断点确认 `isShown()` 与窗口 `mStopped` 状态；复现"未 seal"异常时检查是否自建了 AccessibilityNodeInfo 实例。

根因：目标 View 或宿主窗口不可见/停止（AccessibilityInteractionController.java:776 的复合检查）；自定义 View 只初始化了信息未实现 performAccessibilityAction；对自建未封缄的快照调 getChild 等查询触发 enforceSealed 断言；AMS 三道闸之一被拦（对照第八章 8.2）。

修复：保证目标界面可见后再操作；补齐自定义 View 的动作实现；始终使用服务端下发的快照（已 seal）而非自建实例；按闸的归属分别从能力位、用户身份、AppOps 三个方向恢复。

场景五：dispatchGesture 报手势失败

症状：onPerformGestureResult 回调 completedSuccessfully=false，或手势执行到一半被取消。

定位命令：`adb shell dumpsys accessibility` 确认服务具备手势能力描述；`adb logcat -s MotionEventInjector`（打开 a11y trace 时更详细）；确认设备 `adb shell getprop ro.hardware.type` 场景下是否具备触摸屏。

根因：XML 缺 canPerformGestures 能力位；设备无触摸/假触摸支持（AccessibilityServiceConnection.java:472 检查）；目标显示器没有注入器实例（显示器关闭或输入过滤器未安装）；新手势与在途手势"接不上"被整体拒绝（MotionEventInjector.java:203 的续步校验）；注入过程中真实触摸到来触发取消（讲解注释⑥的设计行为）。

修复：补能力位；避免把注入时段安排在用户实时操作窗口；多指连招按"续步"规则保持同一服务调用方与时间连续性；被取消属正确行为，重试应基于新的屏幕状态。

无障碍核心速查表

机制	一句话	源码锚点	所在册
AMS	AccessibilityManagerService:173 绑定/分发/安全	services/accessibility/	本册
InputFilter	输入链最后消费者（可改写/吞/合成）	AccessibilityInputFilter:59	本册
NodeInfo	View 树语义快照（跨进程一致）	AccessibilityNodeInfo	本册
安全	读屏+合成操作+截胡 = 极高权限	AccessibilitySecurityPolicy	本册
13 限制	禁止 sideload 静默启用 a11y	安装来源限制	本册